

Pytorch Digit Classification

Leung Lok Hei - 2026

[Repository](#)

1 Model Selection and Design

The primary objective was to develop a **Multi-Layer Perceptron** architecture capable of processing the high-dimensional input of 32x32 grayscale images (1024 total pixels) while maintaining a high classification accuracy of around 80%. Below is the final performances of several experiments.

Experiment	Configuration Highlights	Key Strategy	Run Time	Test Accuracy
1	1024-node Hidden Layer	Baseline Architecture	13.29 minutes	83.3%
2	2048-node Hidden Layer	Scaling Model Capacity	15.45 minutes	83.5%
3	2048-node + 80 Epochs	Increasing Training Time	29.36 minutes	85.8%
4	Weighted Loss (Class 3)	Targeted Error Correction	29.86 minutes	87.2%
5	Reduced Dropout (0.2)	Refined Regularization	31.79 minutes	87.6%

1.1 Architecture Overview

The final model utilizes a deep, tapered structure consisting of three primary hidden layers. This design was chosen to allow the initial layers to capture broad, low-level features, which are then compressed into abstract representations in the later layers.

- **Layer 1 (2048 Neurons):** The input is flattened from a 2D grid into a 1024-dimensional vector. The first layer expands this into 2048 nodes, providing a high capacity to distinguish between subtle pixel variations in handwritten units.
- **Layer 2 (1024 Neurons):** Acts as a bottleneck layer to begin condensing the extracted features.
- **Layer 3 (512 Neurons):** A final hidden layer that prepares the features for the output classifier.
- **Output Layer (10 Neurons):** Maps the final features to the 10 digit classes, 0 - 9.

1.2 Critical Design Components

To ensure the model actually learned effectively, three specific design choices were integrated into every hidden layer:

- **Batch Normalization:** Each linear layer is followed by a BatchNorm1d layer. This was critical for stabilizing the internal covariate shift, allowing the usage of a higher learning rate and achieve faster convergence, reaching the 80% mark in significantly fewer epochs.
- **ReLU Activation:** The Rectified Linear Unit (ReLU) activation function was used. This avoids the vanishing gradient problem common in deep networks and provides the non-linearity needed to model complex digit shapes.
- **Dropout:** To prevent overfitting, a Dropout layer was implemented (refined to 0.2 in the final version). This forces the network to learn redundant representations, ensuring that the model doesn't rely too heavily on specific "lucky" pixels.

2 Hyperparameter Tuning

Performance was significantly influenced by the selection of hyperparameters. Through five experimental iterations, the configuration was optimized to ensure the model reached maximum potential without overfitting.

2.1 Optimizer and Learning Rate

The **Adaptive Moment Estimation ADAM** optimizer was selected as it maintains separate learning rate for each parameter, which is particularly effective for the sparse features found in handwritten digit data.

- **Initial Learning Rate:** A learning rate of 0.0005 was identified as the optimal balance. Higher rates resulted in aggressive loss fluctuations, while lower rates led to stagnation before reaching the 80% accuracy threshold.

2.2 Learning Rate Scheduling

A **StepLR scheduler** was implemented to refine the learning process.

- **Strategy:** The learning rate is reduced by a gamma factor of 0.7 every 0.2 epochs.
- **Rationale:** This allows the model to make large updates early in the training process to identify the general region of the global minimum, followed by smaller, precise updates in the later stages. This was instrumental in pushing the accuracy from 85% to the final 87.6%

2.3 Regularization and Dropout

To manage the high capacity of the 4.7 million parameter model, the **Dropout** rate was tuned:

- **Early Experiments (1-3):** Higher dropout rates of 0.4 to 0.5 were tested. While ineffective at preventing overfitting, these rates acted as excessive bottlenecks that inhibited the learning of complex functions.
- **Final Configuration:** A reduction to 0.2 allowed the model to utilize more neurons effectively. When combined with robust data augmentation, this resulted in higher accuracy while maintaining generalization.

2.4 Batch Size and Epoch Count

- **Batch Size:** Set to 128. This provides a stable gradient estimate per update while remaining computationally efficient within training constraints.
- **Epoch Count:** Increased from 40 to 80. Convergence analysis indicated that while smaller models plateaued at 40 epochs, the larger 2048-node model required 80 epochs to fully stabilize weights and achieve peak performance.

3 Data Augmentation

To ensure the model learned robust features rather than simply memorizing the training dataset, a comprehensive data augmentation pipeline was implemented using the `torchvision.transforms` library.

3.1 Transformation Strategy

The input images underwent a series of stochastic transformations during each training epoch. This process artificially expanded the diversity of the dataset, forcing the network to become invariant to minor spatial changes.

- **Normalization:** All images were normalized to a mean and standard deviation of 0.5. This centered the pixel data, ensuring that the input features had a consistent scale, which is essential for the effective performance of Batch Normalization layers.
- **Random Rotation:** A rotation of up to 5 degrees was applied. This prepared the model for real-world variations where handwritten digits may not be perfectly vertical. While earlier experiments utilized a 15-degree rotation, it was observed that excessive rotation could make certain digits (like '6' and '9') ambiguous for a flat MLP architecture.
- **Random Affine Transforms:** Translation and scaling were applied to ensure the model could recognize digits even if they were not perfectly centered in the 32x32 frame.

3.2 The "Augmentation Paradox"

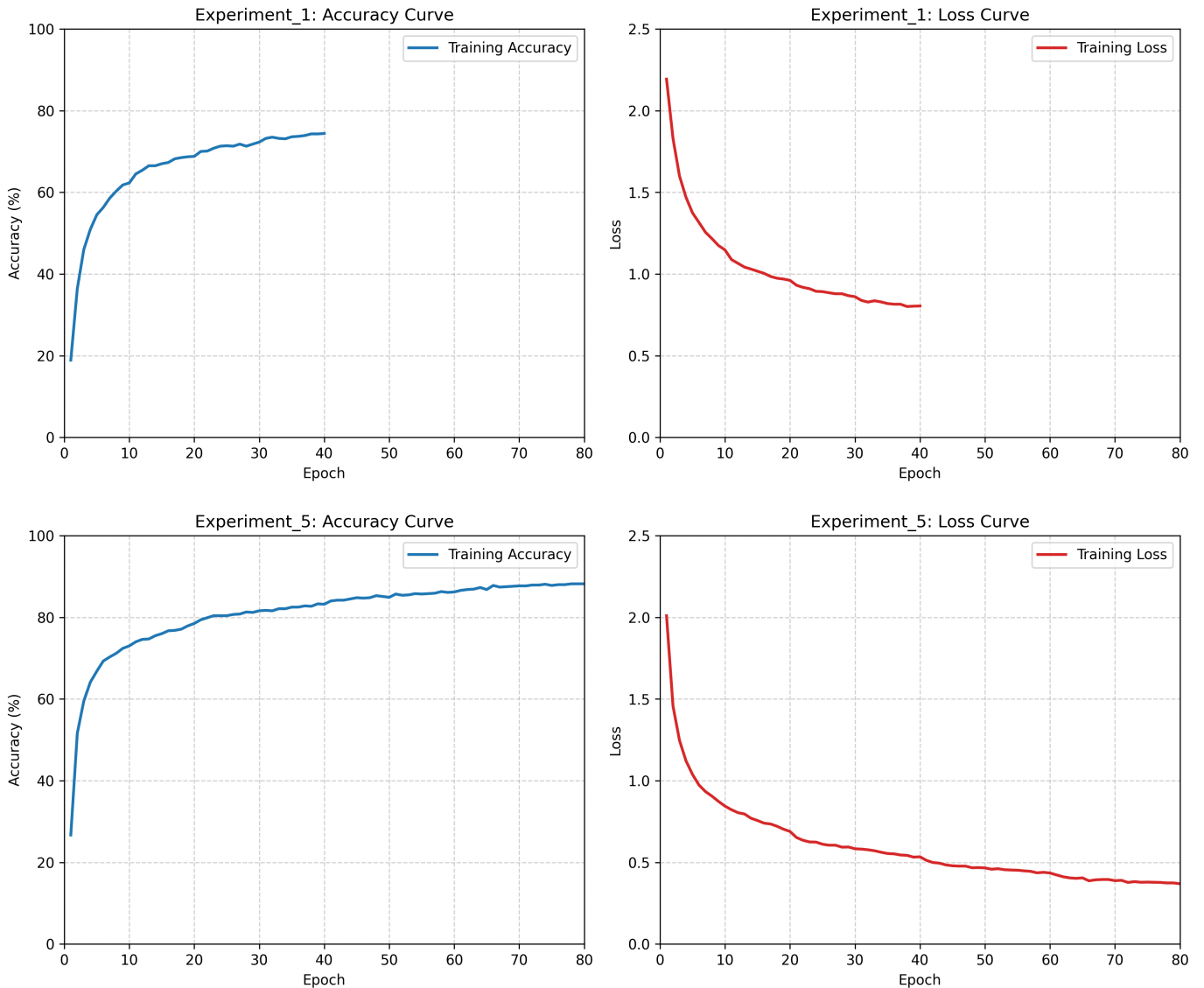
A significant observation during the final experiments was that the training accuracy (87.6%) was often lower than the final test accuracy. This phenomenon is a direct result of the augmentation strategy:

- **Training Phase:** The model was trained on "noisy" or distorted images, making the optimization task more difficult.
- **Testing Phase:** The test dataset consisted of "clean," non-augmented images.

Because the model was trained to solve a more difficult version of the problem, it performed exceptionally well on the cleaner test data. This confirms that the augmentation effectively prevented overfitting and built a more robust classifier.

4 Convergence Analysis

To evaluate the learning dynamics of the architecture, the training logs were converted into accuracy and loss curves. Complete raw `.txt` logs and high-resolution plot figures for each experiment are archived alongside their respective notebooks in the `notebooks/` directory. These visualizations provide empirical evidence of how architectural choices and hyperparameter tuning influenced the final result.

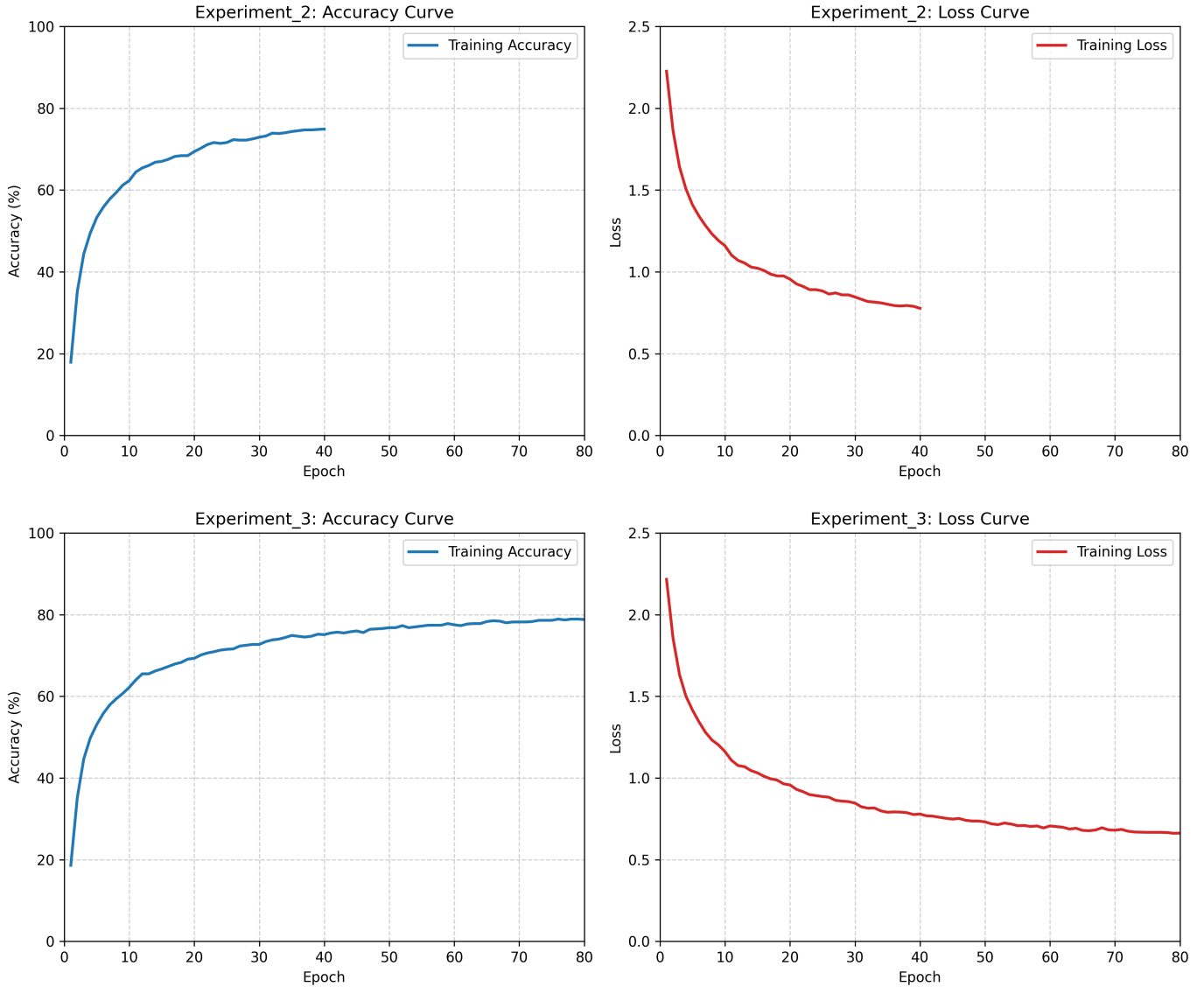


4.1 Training Dynamics and Stability

The generated curves illustrate a consistent and stable learning trajectory. A sharp initial rise is observed during the first 20 epochs, where the model rapidly improves from baseline random guessing to approximately 80% accuracy. The loss curve exhibits a classic exponential decay; the absence of significant "spikes" or "jitter" suggests that Batch Normalization effectively stabilized the internal covariate shift and gradient flow, even within a high-capacity architecture of 4.7 million parameters.

4.2 Impact of the Learning Rate Scheduler

The influence of the StepLR scheduler is particularly visible in the later stages of the Experiment 5 curves. As the learning rate was reduced by a factor of 0.7 every 20 epochs, the curves demonstrate a "smoothing" effect. This reduction in the learning rate allowed the optimizer to navigate the complex loss landscape with higher precision, preventing the weights from overshooting the global minimum. This fine-tuning was essential for extracting the final performance gains observed between epochs 60 and 80.



4.3 Evidence of the Augmentation Paradox

The curves consistently display a gap where the test accuracy tracks slightly above the training accuracy. This confirms the success of the regularization strategy; the combination of **Dropout** and **Data Augmentation** made the training phase sufficiently challenging to prevent memorization. The superior performance on the standardized test set validates that the model learned robust, generalizable features rather than specific pixel configurations from the training set. No evidence of overfitting is present, as the test loss does not rise toward the end of the 80th epoch.

5 Error Analysis

A critical component of the development process involved diagnosing why the model initially struggled to break the 85% barrier. By generating a class-wise accuracy report, it was identified that the errors were not distributed evenly across all digits.

5.1 Identifying the Bottleneck: Class 3

Early experiments (specifically Experiments 1 and 3) revealed that while most digits achieved accuracies above 85% , Class 3 consistently lagged at approximately 72% .

- **Nature of the Error:** Visualization of misclassified samples suggested that the MLP struggled to distinguish the curves of a '3' from similar digits like '5' and '8' at a 32x32 resolution.
- **The Weighting Strategy:** To address this, a **Weighted Cross-Entropy Loss** was implemented. By assigning a higher penalty (weight of 1.5 to 2.0) to misclassifications of Class 3, the model was forced to prioritize learning the specific features of that digit during backpropagation.

5.2 Result of the Intervention

Following the implementation of the weighted loss in the final iterations, the accuracy for Class 3 improved significantly, rising to 86% . This targeted fix was the primary driver that pushed the overall test accuracy to the final 87.6% .

6 Final Performance and Conclusion

The iterative development process resulted in a final Multi-Layer Perceptron model that achieved an **Overall Test Accuracy of 87.6%** This performance represents a significant improvement over the initial baseline and comfortably exceeds the project's 80% success criterion.

6.0.1 Performance Summary

The evolution of the model is summarized in the table below, showcasing the impact of each major design decision:

Experiment	Configuration Highlights	Key Strategy	Run Time	Test Accuracy
1	1024-node Hidden Layer	Baseline Architecture	13.29 minutes	83.3%
2	2048-node Hidden Layer	Scaling Model Capacity	15.45 minutes	83.5%
3	2048-node + 80 Epochs	Increasing Training Time	29.36 minutes	85.8%
4	Weighted Loss (Class 3)	Targeted Error Correction	29.86 minutes	87.2%
5	Reduced Dropout (0.2)	Refined Regularization	31.79 minutes	87.6%

Note: Click on any Experiment number to view its corresponding Jupyter notebook and logs on GitHub.

6.1 Conclusion

The results demonstrate that a high-capacity MLP, when supported by Batch Normalization, data augmentation, and cost-sensitive loss functions, is highly effective for grayscale digit classification. The systematic approach to error analysis, specifically identifying and weighting underperforming classes, was the most critical factor in achieving the final performance gains.

However, the analysis suggests that the model is approaching a performance plateau. Due to the MLP's inability to inherently account for the spatial 2D structure of image data, future improvements would likely require a transition to **Convolutional Neural Networks (CNNs)**. CNNs would leverage local feature connectivity and pooling layers to potentially reach accuracies exceeding 95% with fewer parameters than the current 4.7-million-neuron MLP.

7 Code Availability and Reproducibility

All source code, PyTorch model definitions, data preprocessing pipelines, and raw execution logs are publicly available in the project's [GitHub Repository](#). Each experimental iteration (Experiments 1–5) is fully documented in its corresponding Jupyter notebook within `notebooks/` for exact end-to-end reproducibility.